Advanced Data Structures

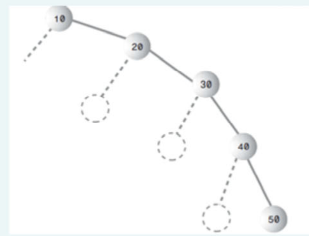
Balanced Search Trees

- Red-black trees
- Height of a red-black tree
- Rotations
- Insertion

Balanced and Unbalanced Trees

Binary Trees are balanced when the left and the right subtrees are almost at the same level.

Trees become unbalanced when you start inserting nodes either in ascending and descending order.



When a tree is unbalanced and there are no branches, it becomes a list.

Speed of searching is reduced from O(log n) to O(n).

What will happen to the insert and delete efficiency ?

Balanced search trees

Balanced search tree: A search-tree data structure for which a height of $O(\lg n)$ is guaranteed when implementing a dynamic set of n items.

- Red-Black trees
- AVL trees
- 2-3 trees
- 2-4 trees
- B-trees

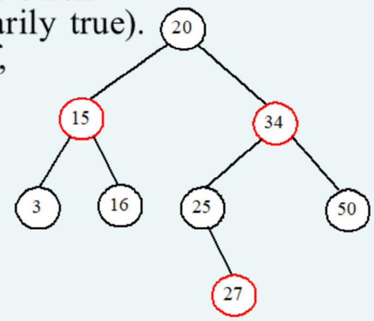
Red black trees

Characteristics:

- The nodes are colored
- During insertion and deletion, rules are followed that preserve the various arrangements of these colors.
- Ensures that the tree is always balanced thus preserving the efficiency.

Red-Black rules

1. Every node is either red or black
2. The root is always black.
3. If a node is red, its children must be black (although the converse isn't necessarily true).
4. Every path from the root to the leaf, or an external node, must contain the same number of black nodes. (black depth)



A node is called *external* if its left or right subtree is empty. Note that a leaf node is external, but an external node is not necessarily a leaf node. For example, node 25 is external, but it is not a leaf. The *black depth* of a node is defined as the number of black nodes in a path from the node to the root. For example, the black depth of node 25 is 2 and the black depth of node 27 is 2.

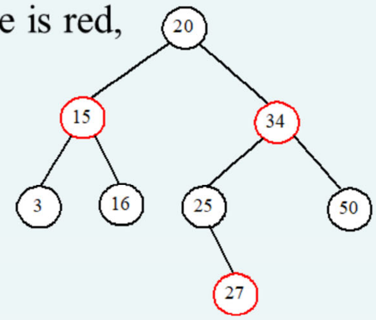
Modifying operations

The operations INSERT and DELETE cause modifications to the red-black tree:

- the operation itself,
- color changes,
- restructuring the links of the tree via ***"rotations"***.

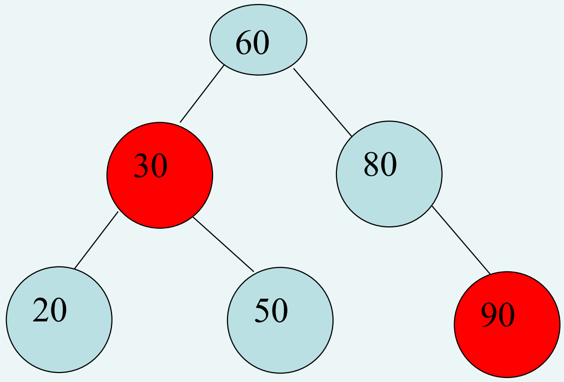
How to insert an element to red-black tree.

- A new node is always inserted as a leaf node.
  - If root, color it black, otherwise, color it red.
  - If the parent of the new node is red, it violates property 2 of the red-black tree. (double-red violation)



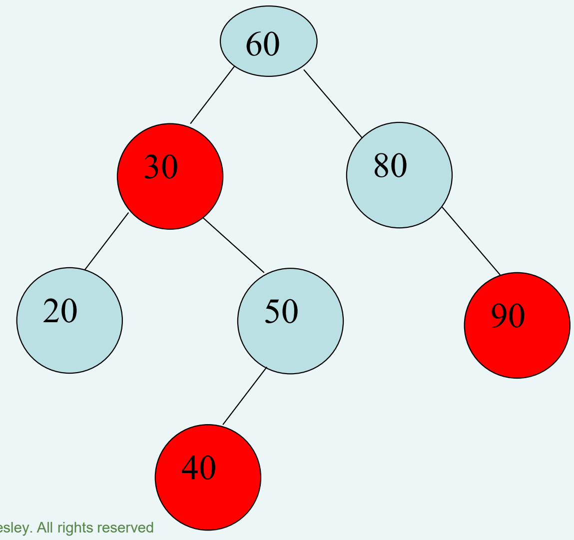
Insert 26.

Is the following a RB-Tree?

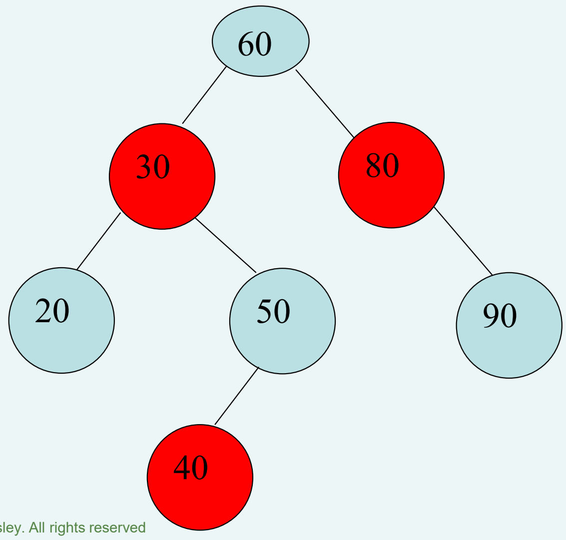


13 B-8

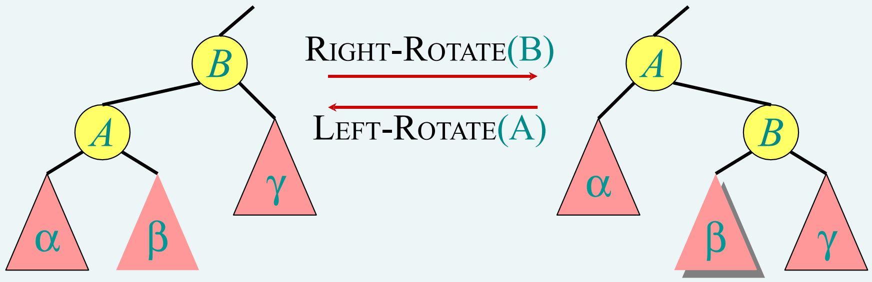
Is the following a RB-Tree?



Is the following a RB-Tree?



Rotations



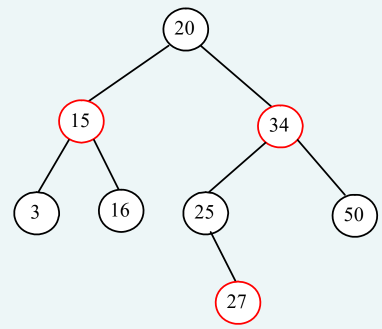
Rotations maintain the inorder ordering of keys:

- $a \in \alpha$, $b \in \beta$, $c \in \gamma \Rightarrow a \le A \le b \le B \le c$.

A rotation can be performed in $O(1)$ time.

How to insert an element to red-black tree.

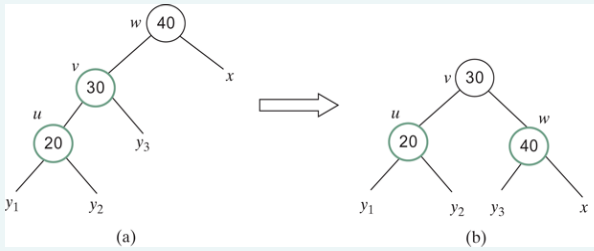
- Let u denote the new node inserted, v the parent of u, w the parent of v, and x the sibling of v.
- To fix the double-red violation, consider two cases:
  - Case 1: x is black or x is null.
    - There are four possible configurations for u, v, w, and x



13 B-12

Inserting 26 violates rule 2

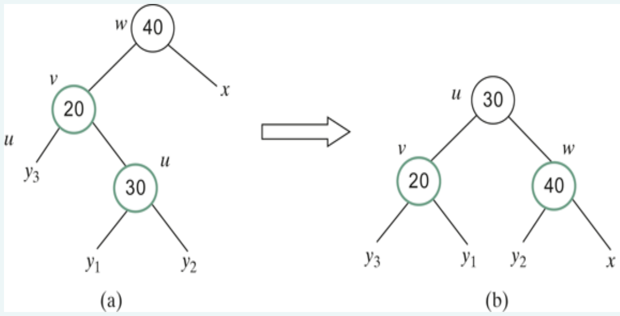
Case 1.1: u<v<w



Flip colors → Rotate right
V ← black
W ← red
Rorate right

Rotate right, flip color

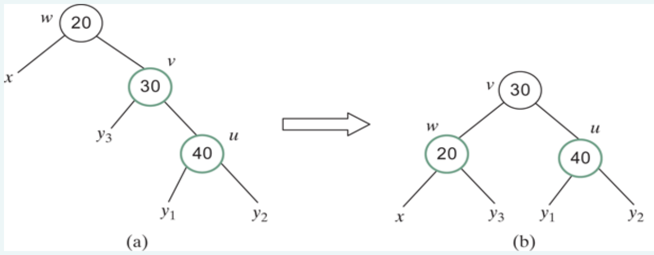
Case 1.2: v<u<w



Flip colors → double rotaion (LR)
U ← black
W ← red
Rotate left → Rotate right

LR double rotate, flip colors

Case 1.3: w<v<u

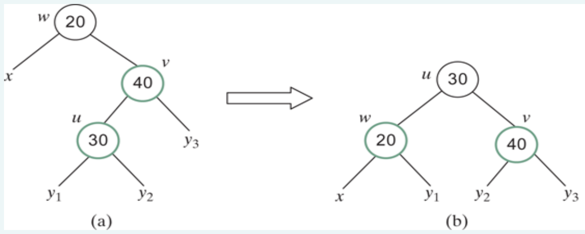


Flip color → Rotate left
V ← black
W ← red
Rotate Right

13 B-15

Rotate left. Flip color

Case 1.4: w<u<v

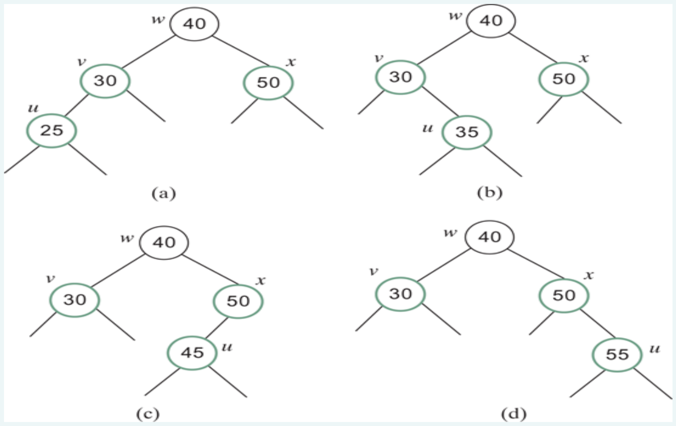


Flip colors → Double rotation (RL)
W ← red
U ← black
Rotate right → Rotate left

RL double rotate, flip color

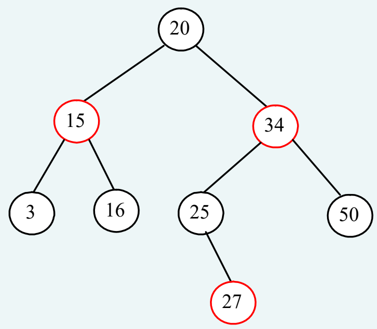
- Case 2: x is red.
  - There are four possible configurations for u, v, w, and x.

Case 2 has four possible configurations.



Flip colors
X & V ← black
W & X ← red

Insert 26



 13 B-19

Violates rule 2

Recolor w → red

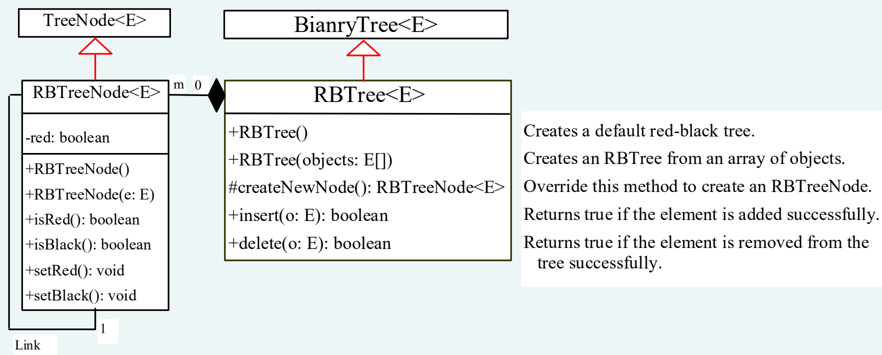
Recolor u → black

Double rotation

Right around 27

Left around 25

Designing Classes for Red Black Trees



20